Smart Irrigation and Soil Monitoring System

A Project Report Submitted in Partial Fulfillment of
the requirements for theAward of Degree of

BACHELOR OF TECHNOLOGY(B.TECH)

BY

R.Trinadh(2300040129).

P.Venkatesh (2300040159).

G.Arvind (2300040229).

P.Rupendra(2300040464).

Under esteemed guidance of

PROF A.V PRABHU

K L University

u/s 3 of UGC Act, 1956

Koneru Lakshmaiah Education Foundation

Green Fields, Vaddeswaram, Guntur,

AndhraPradesh-522302

2026-2027

KLEF Deemed to be university,

Green fields vaddeswaram,

Guntur district,A.P.,INDIA..

BONAFIDE CERTIFICATE

Certified that this project report titled **Smart Irrigation and Soil Monitoring System**
is the bonafide work of **R.Trinadh . P.Venkatesh (2300040159), G.Arvind (2300040229),P.Rupendra(2300040464).** Who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Abstract

Smart Irrigation and Soil Monitoring System

Water management is one of the major challenges in agriculture. Traditional irrigation methods often lead to overwatering or underwatering, which affects crop yield and wastes water. The Smart Irrigation and Soil Monitoring System is designed to automate the irrigation process by monitoring soil conditions in real time.

The system uses a soil moisture sensor to measure the water content present in the soil. A microcontroller such as Arduino or ESP32 processes the sensor data and compares it with a predefined threshold value. If the soil moisture level falls below the set limit, the controller automatically activates a water pump through a relay module. Once the soil reaches the required moisture level, the pump is turned off automatically. Additional sensors like temperature and humidity sensors can also be integrated to enhance monitoring accuracy.

This system reduces water wastage, minimizes human effort, and improves crop productivity. It can also be integrated with IoT technology to allow remote monitoring and control through a mobile application or web platform. The proposed system provides an efficient, low-cost, and reliable solution for smart farming and sustainable agriculture.

HARDWARE AND SOFTWARE COMPONENTS.

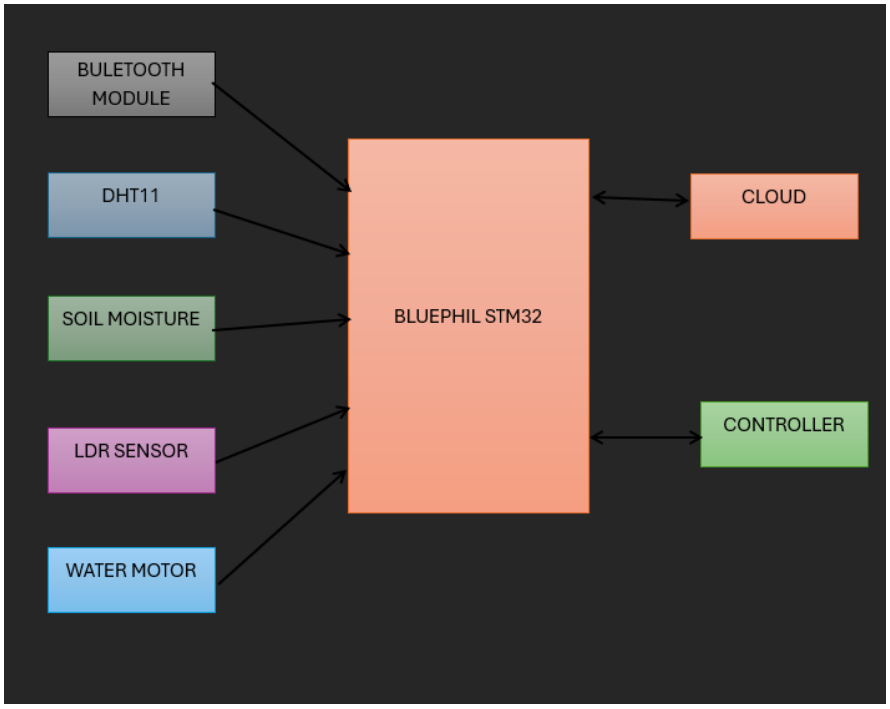
HARDWARE COMPONENTS:

- **BluePhil Board**
- **Connectivity Module**
- **Soil Moisture Sensor**
- **Temperature & Humidity Sensor (DHT11/DHT22)**
- **Relay Module**
- **Water Pump / DC Motor**
- **Power Supply (5V / 12V)**
- **Connecting Wires**

SOFTWARE COMPONENTS:

- **Arduino IDE**
- **Google collab**

BLOCK DIAGRAM



1. Soil Moisture Sensor

The soil moisture sensor is used to measure the water content present in the soil. It continuously monitors the moisture level around the crop roots and generates an electrical signal based on the soil condition. When the soil is dry, the sensor produces a low moisture value, and when sufficient water is present, it produces a higher value. This sensor helps in determining the exact water requirement of the crops and prevents both under-irrigation and over-irrigation.

---

2. Temperature and Humidity Sensor

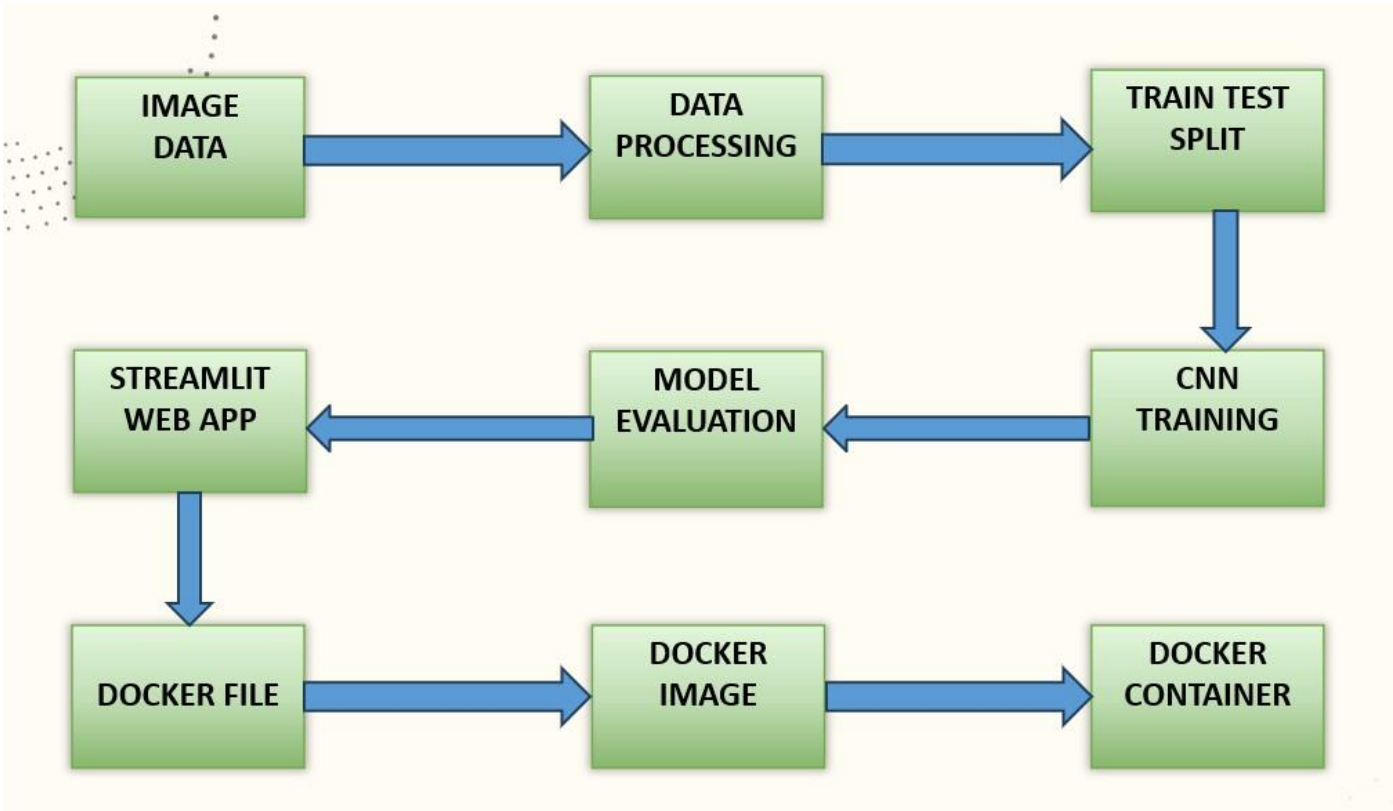
The temperature and humidity sensor monitors the surrounding environmental conditions of the agricultural field. It measures the ambient temperature and relative humidity, which are important factors affecting crop growth and disease occurrence. These parameters help the system in understanding weather conditions and adjusting irrigation accordingly. High temperature and low humidity conditions indicate increased water demand for crops.

3.The Blue Pill board is a low-cost development board based on the STM32F103C8T6 microcontroller. It features a 32-bit ARM Cortex-M3 processor running at 72 MHz, which makes it faster and more powerful than many basic 8-bit boards. The board operates at 3.3 V and provides multiple GPIO pins, 12-bit ADC channels, timers, PWM outputs, and communication interfaces such as UART, SPI, and I2C. Due to its small size, low power consumption, and high performance, the Blue Pill is widely used in embedded systems, automation, robotics, and IoT projects. It can be programmed using STM32CubeIDE, Keil, or even the Arduino IDE with proper configuration, making it suitable for both beginners and advanced embedded developers.

4.The relay module is used to safely control high-power devices such as the water pump using low-power signals from the ESP32. Since the ESP32 operates at low voltage levels, it cannot directly drive the motor. The relay provides electrical isolation between the control circuit and the power circuit, ensuring safe and reliable operation of the irrigation system.

5.Water Pump The water pump is responsible for supplying water to the crops during irrigation. When the ESP32 detects that the soil moisture level is below the required threshold, it activates the relay, which turns ON the water pump. The pump supplies water through pipes or drip irrigation until the soil reaches the desired moisture level, after which the pump is automatically turned OFF.

WORK FLOW



1. Image Data

Image data forms the foundation of the crop disease detection system. It consists of leaf images collected from agricultural fields or public datasets. These images include both healthy and diseased crop leaves. The quality and diversity of the image dataset play a crucial role in improving the accuracy of the machine learning model. The images serve as input for training and testing the disease classification system.

---

2. **Data Processing**

Data processing involves preparing raw images for machine learning analysis. This step includes image resizing, normalization, noise removal, and data augmentation techniques such as rotation, flipping, and scaling. These operations improve model robustness and help reduce overfitting. Data processing ensures that all images are in a consistent format suitable for CNN-based training.

---

3. Train–Test Split

The processed dataset is divided into training and testing subsets. The training set is used to train the convolutional neural network, while the testing set is used to evaluate model performance on unseen data. This separation helps in assessing the generalization capability of the model and prevents biased performance evaluation.

---

4. CNN Training

Convolutional Neural Network (CNN) training is the core stage of the disease detection system. The CNN automatically extracts important features such as color variations, leaf texture, and disease patterns from the images. During training, the model learns to associate these features with corresponding disease classes by minimizing loss and improving accuracy through multiple training epochs.

---

5. Model Evaluation

Model evaluation is performed to assess the effectiveness of the trained CNN. Performance metrics such as accuracy, precision, recall, and F1-score are calculated using the test dataset. This step ensures that the model is reliable and capable of correctly identifying crop diseases in real-world conditions.

---

6. Streamlit Web Application

The Streamlit web application provides an interactive user interface for disease detection. Users can upload crop leaf images through the web interface, and the trained CNN model predicts the disease class in real time. This application makes the system user-friendly and accessible to farmers and agricultural experts without requiring technical knowledge.

---

7. Docker File

The Dockerfile defines the software environment required to run the Streamlit application and the trained machine learning model. It specifies dependencies, libraries, and system configurations. Using a Dockerfile ensures consistency across different deployment platforms and eliminates environment compatibility issues.

---

8. Docker Image

The Docker image is a packaged version of the application created using the Dockerfile. It contains the trained CNN model, Streamlit web application, and all necessary dependencies. This image enables easy distribution and deployment of the system on any Docker-supported platform.

9. Docker Container

The Docker container is a running instance of the Docker image. It executes the crop disease detection application in an isolated and controlled environment. Containers ensure scalability, portability, and efficient resource utilization, making the system suitable for real-world deployment.

The proposed workflow describes a systematic approach for detecting crop diseases using image-based machine learning techniques. The system integrates data acquisition, deep learning-based image classification, and web-based deployment to provide an efficient and scalable solution for agricultural disease monitoring.

The workflow begins with the collection of image data consisting of healthy and diseased crop leaf images. These images capture visible symptoms such as discoloration, spots, and texture variations, which are critical indicators of plant diseases. High-quality and diverse image datasets improve the learning capability of the model and enhance detection accuracy.

In the data processing stage, the collected images are preprocessed to make them suitable for deep learning analysis. Preprocessing includes resizing images to a fixed resolution, normalizing pixel values, and applying data augmentation techniques such as rotation and flipping. These steps help reduce noise, handle variations in image conditions, and prevent overfitting during model training.

CODE

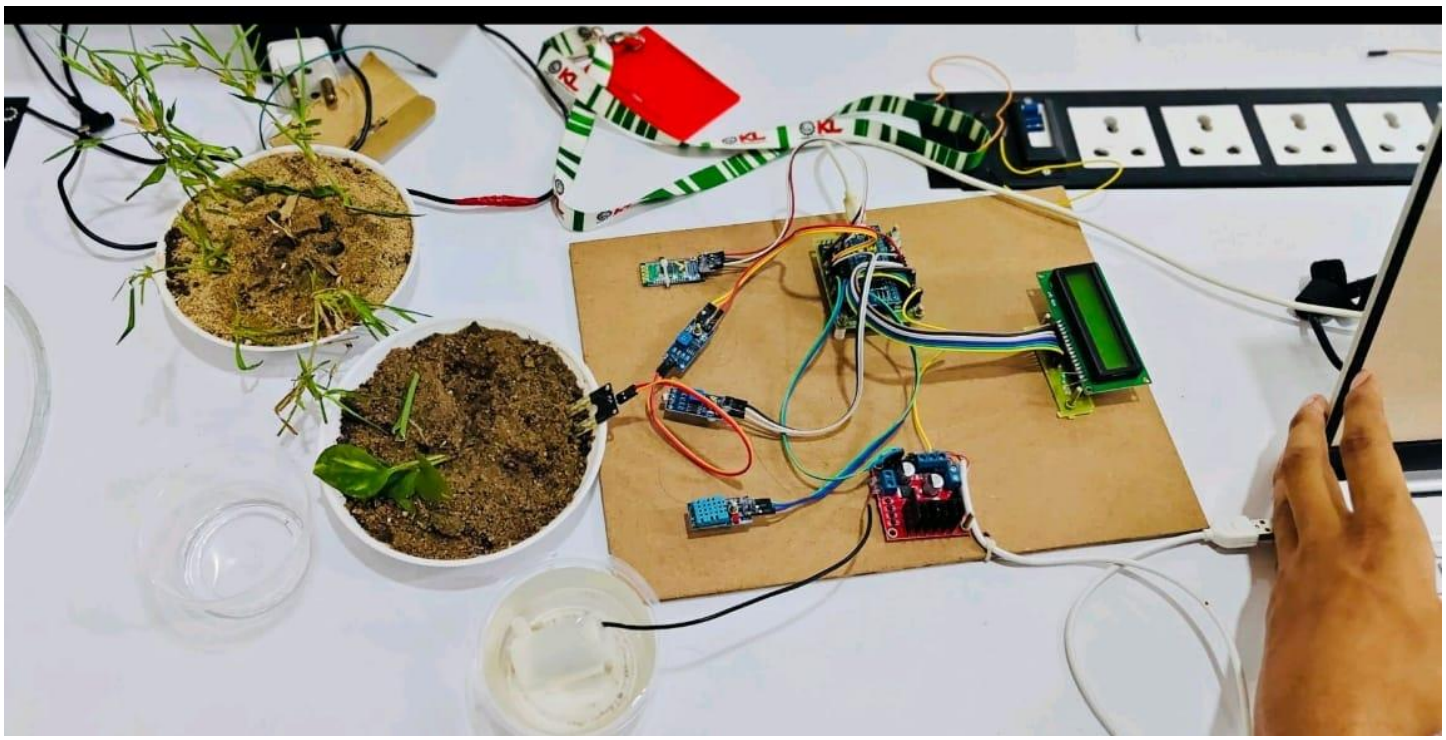
```
#include <LiquidCrystal.h>
const int rs =PC14, en =PC15, d4 =PA0, d5 =PA1, d6 =PA2, d7 =PA3;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);
int mot=PB7;
int cnt=0;
int x=0;
int pos = 0;
int t,h;
int md=0;
int ms=PA5;
int ldr=PA6;
#include "DHT.h"
#define DHTPIN PB8
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
lcd.begin (16,2);
lcd.setCursor(4,0);
lcd.print("WELCOME");
lcd.setCursor(0,1);
lcd.print("SMART AGRICULTURE:");
delay(2000);
lcd.clear();

Serial.begin(9600);
dht.begin();
```

```
pinMode(ms,INPUT);
pinMode(ldr,INPUT);
pinMode(mot,OUTPUT);




}
void loop() {
  cnt++;
int h = dht.readHumidity();
int t = dht.readTemperature();
int mval=1023-analogRead(ms)/4;
int lval=1023-analogRead(ldr)/4;
lcd.clear();
lcd.setCursor(0,0);
lcd.print("T:");
lcd.setCursor(2,0);
lcd.print(t);
lcd.setCursor(5,0);
lcd.print("H:");
lcd.setCursor(7,0);
lcd.print(h);
lcd.setCursor(0,1);
lcd.print("M:");
lcd.setCursor(2,1);
lcd.print(mval);
lcd.setCursor(7,1);
lcd.print("L:");
```

```
lcd.setCursor(10,1);
lcd.print(lval);
delay(1000);
if (Serial.available()){
  int x=Serial.read();
  if (x=='1'){
    lcd.setCursor(11,0);
    lcd.print("AUTO");
    md=0;
  }
   if (x=='2'){
      lcd.setCursor(11,0);
    lcd.print("MANU");
     digitalWrite(mot,0);
    md=1;
  }
   if (x=='3'){
    digitalWrite(mot,1);

  }
  if (x=='4'){
    digitalWrite(mot,0);

  }
}
if (md==0){

if( mval>400)
{
   digitalWrite(mot,0);
```

```
  }
else
{
  digitalWrite(mot,1);


}
}

Serial.println("temp:"+String(t)+" hum:"+String(h) + " lig:"+String(lval)+" mos:"+String(mval));
delay(1000);
}
```

OUTPUT



TEAM MEMBER DETAILS

R.Trinadh(2300040129).

P.Venkatesh (2300040159).

G.Arvind (2300040229).

P.Rupendra(2300040464).